

Tutti: 实现基于 SSD 的 KV 缓存用于长上下文大模型服务

Shi Qiu*

Xiamen University
Xiamen, China

Yifan Hu

Xiamen University
Xiamen, China

Xintao Wang

Shanghai Jiao Tong University
Shanghai, China

Wenhao Zhu

Xiamen University
Xiamen, China

Jianqin Yan

Xiamen University
Xiamen, China

Hao Chen

Xiamen University
Xiamen, China

Kaiqiang Xu

Hong Kong University of Science and
Technology
Hong Kong, China

Kai Chen

Hong Kong University of Science and
Technology
Hong Kong, China

Yiming Zhang[†]

Shanghai Jiao Tong University
Shanghai, China

摘要

大语言模型服务依赖前缀缓存来提升推理性能。随着上下文不断增长,键值(KV)缓存的占用空间远远超出 GPU HBM 和 CPU DRAM 的容量,因此 KV 缓存越来越多地被卸载到 NVMe SSD 上。然而,从 SSD 恢复 KV 缓存面临糟糕的 I/O 性能,并导致显著的 GPU 停顿。这主要是因为碎片化的 GPU 内存布局导致了大量微小的随机 I/O,即使使用 GPU 直接存储 (GDS),由于仍需 CPU 介入发起每次 I/O,使得低并行性的 CPU 成为严重瓶颈,因而仍然为 *CPU-centric*。

本文提出了 *Tutti*, 一种高效的基于固态硬盘 (SSD) 的键值(KV)缓存解决方案,该方案从高带宽内存(HBM)与固态硬盘(SSD)之间的关键数据和 I/O 控制路径中彻底消除了 CPU 的干预。*Tutti* 的核心是一个以 GPU 为中心的 KV 缓存对象存储系统,其中 CPU 仅需在每层中异步加载一次 I/O 内核至 GPU。通过以下设计, *Tutti* 充分利用了 NVMe SSD 的带宽,并将 GPU 停顿降至接近零: (i) 我们提供了一种原生支持 GPU 的对象抽象机制,实现了批量 KV 缓存传输与管理; (ii) 通过引入 GPU io_uring, 重构了 GPU 存储栈,以支持异步的 GPU 直接对象 I/O; (iii) 我们提出了感知空闲资源的 I/O 调度策略,以避免 GPU 资源争用。我们已实现 *Tutti* 并将其集成至 vLLM。大量评估表明,相较于最先进的支持 GDS 的、基于 SSD 的 LMCACHE, *Tutti* 在严格服务等级协议 (SLO) 约束下将首字节时间 (TTFT) 降低了 78.3%, 并使可达到的请求速率提升 2 $\times$ 。服务成本降低了 27%。*Tutti* 实现了几乎与基于 DRAM 的 LMCACHE 相同的推理性能,同时提供了近乎无限的容量。

1 引言

大语言模型 (LLMs) 正在将数据中心从数据存储平台转变为人工智能服务的 token 生成基础设施 [31, 56]。对于模型即服务 (MaaS) 提供商而言, token 生成的延迟和

成本决定了服务的竞争力。前缀缓存 [6, 38, 50] 已成为现代推理服务中的关键最优化技术。它重用之前计算过的 token, 即所谓的 键值 (KV) 缓存, 以避免冗余计算, 从而提升服务等级目标 (SLO), 并将每个 token 的成本降低多达一个数量级 [7, 36]。

随着大语言模型的上下文窗口和并发量不断增长, KV 缓存的占用迅速增加。GPU HBM 容量很快被耗尽, 导致必须进行 KV 淘汰和重新计算, 从而增加延迟和成本, 同时限制了 MaaS 服务商能够维持的并发会话数量 [9]。通常使用 CPU DRAM 来扩展超出 HBM 的 KV 容量, 但在大规模场景下仍显不足。例如, 即使拥有约 2 TB 的 DRAM, 也仅能保留大约五分钟的 KV 缓存 [35]。因此, 进一步扩展需要引入 NVMe SSD 作为下一层级存储 [12, 13, 26, 27, 38, 44, 51, 54]。商用服务器可提供超过 100 TB 的 NVMe SSD 容量 [9], 足以支持长时间运行的对话以及新兴的 Agentic 工作负载, 保留超过一小时的 KV 缓存。

然而, 三层次的 HBM-DRAM-SSD 键值 (KV) 缓存系统对于延迟敏感的大型语言模型 (LLM) 推理而言过于缓慢。瓶颈并非 SSD 的原始带宽 [18, 45], 而是由现代 LLM 引擎 (vLLM [20] 和 SGLang [42]) 采用的细粒度、基于页的 GPU 内存布局所导致, 该布局将逻辑上连续的 KV 缓存分割成大量小而分散的块 [19, 26, 32, 57]。从 SSDs 恢复一个长前缀会生成大量微小的随机 I/O 操作 [26, 50], 再加上 DRAM-HBM 之间的数据复制以及 CPU-GPU 间的同步开销, 进一步加剧了性能损耗。所有这些操作都需要 CPU 介入, 因此三层次的 KV 缓存层级结构是以 CPU 为中心的 [39]。综合来看, 这些开销显著降低了 SSD 到 GPU 的有效带宽, 并导致 70~80% 的 GPU 停顿 [41]。大量的昂贵 GPU 周期被浪费在等待 KV 缓存从 SSD 传输至 HBM (通过 DRAM) 的过程中, 使得 KV 缓存重用甚至比重新计算还要慢 [6, 13, 16, 17, 37]。

一种常见的最优化方法是将键值 (KV) 缓存传输与计算流水线化, 以缓解传输开销, 这种方法对基于 DRAM 的 KV 缓存系统有效 [13, 16]。然而, 在 SSD 上, 流水线化会导致传输碎片化, 降低有效带宽, 并引入额外的 CPU

*qiushijxs@outlook.com

[†]Yiming Zhang is the corresponding author. sdiris@gmail.com

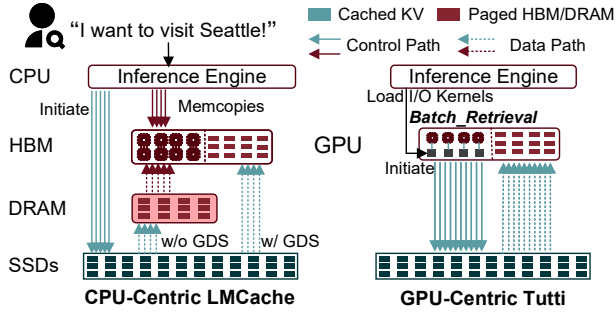


图 1. 与以 CPU 为中心的 KV 缓存存储 (LMCache 带和不带 GDS) 相比, 以 GPU 为中心的全图消除了 CPU 在 HBM 与 SSDs 之间关键数据和 I/O 控制路径中的介入。

端调度和控制开销, 从而进一步恶化 I/O 性能。因此, 现有系统倾向于避免将 KV 缓存卸载到 SSD, 而将大部分 KV 缓存保留在 DRAM [1, 11, 21, 38, 50, 55] 中, 其有限的容量降低了命中率, 削弱了前缀缓存的优势。

最先进的 LMCache [26] 将 GPU 直接存储 (GDS) [33] 集成到其键值 (KV) 缓存层次结构中, 实现了 (可选的) 两级 HBM-SSD 模式, 支持 GPU 与 SSD 之间的直接访问。然而, 由于每次 I/O 操作必须由 CPU (图 1(左)) 发起, GDS 仍然保持以 CPU 为中心, CPU 仍处于关键的 I/O 控制路径上。因此, 即使启用了 GDS 的 LMCache 在将 KV 缓存从 HBM 传输到 SSD 时仍会遭遇 I/O 瓶颈 (§2.2)。随着 GPU 计算能力和模型侧效率持续提升, 这一问题进一步加剧。

本文提出了 Tutti, 一种高效的基于 SSD 的 KV 缓存解决方案, 它消除了从高带宽内存 (HBM) 到 SSD 之间的关键数据和 I/O 控制路径中的 CPU 干预 (图 1(右))。Tutti 的核心是一个以 GPU 为中心的、两级 (HBM-SSD) KV 缓存对象存储, 其中 CPU 仅负责在每层上异步加载一次 I/O 内核到 GPU, 将 CPU 开销从 $O(\text{layer} \times \text{blocks})$ 降低至 $O(\text{layer})$ 。这使得 CPU 不再成为瓶颈, 从而让 GPU 能够直接向 SSD 发起大量并行的 I/O 请求来获取 KV 缓存对象。

尽管已探索了以 GPU 为中心的存储用于原始块 (BaM [40]) 和文件 (GeminiFS [39] 和 GoFS [23]), 但将其扩展到 KV 缓存场景仍面临挑战 (§2.4), 原因包括: (i) KV 缓存管理中的抽象不匹配, (ii) KV 缓存传输与 GPU 存储 I/O 之间的粒度差距, 以及 (iii) GPU 资源竞争。Tutti 通过以下设计解决了这些挑战, 从而充分利用 NVMe SSD 带宽, 并将 GPU 停顿减少至接近零。

首先, 我们提供了一种 GPU 原生的对象抽象 (§3.1), 该抽象支持批量的键值缓存传输与管理, 使 GPU 能够直接访问存储在 NVMe SSD 上的键值缓存。为实现这一目标, 我们引入了一个 GPU 文件池、一个基于 GPU 文件系统 (如 GeminiFS) 的 NVMe 文件池, 以及一个 P2P 内存映射表。此外, 我们还公开了一个 CPU 端接口, 将分配、索引和高并发 GPU 访问整合为单一操作。

其次, 我们重新设计了 GPU 存储栈 (§3.2), 以支持异步 GPU 直接对象 I/O。具体而言, 我们引入了 GPU

io_uring (gio_uring), 它模拟了 CPU 端的 io_uring 机制, 将 I/O 提交和完成从 GPU 计算的关键路径中移除。我们对 GPU 资源进行划分, 使得 I/O 内核与计算内核能够并行运行。

第三, 我们提出了感知松弛的 I/O 调度 (§3.3), 以避免 GPU 资源争用, 从而提升端到端推理性能。我们通过离线分析估算每层的 I/O 松弛时间, 并在这些松弛时间内调度 KV 缓存传输, 以最大化计算与 I/O 的重叠, 最小化 GPU 停顿。

我们已实现 Tutti 并将其集成到 vLLM [20] (§3.4)。大量评估表明, 与最先进的基于 SSD 的 LMCache (使用 GDS) 相比, 在严格的 SLO 约束下, Tutti 将 TTFT 降低了 78.3%, 并将可达到的请求率提升了 2×。服务成本降低了 27%。

本文做出了以下贡献:

- 据我们所知, Tutti 是首个开源的基于 SSD 的键值缓存解决方案, 它消除了 HBM 与 SSD 之间的关键数据和 I/O 控制路径中对 CPU 的依赖。
- 我们提供一种 GPU 原生的对象抽象, 以弥合 KV 缓存传输与 GPU 存储 I/O 之间的粒度差距, 同时结合异步 GPU io_uring 和松弛感知的 I/O 调度。
- 我们将 Tutti 集成到 vLLM 中, 展示了其在饱和 NVMe SSD 带宽并几乎消除 GPU 停顿方面的有效性。基于 SSD 的 Tutti 实现了与基于 DRAM 的 LMCache 几乎相同的推理性能, 同时提供了几乎无限的容量。

2 背景与动机

本节首先介绍大语言模型推理中 token 生成与键值缓存 (KV cache) 的基础知识。接着, 我们指出现有分层存储在 KV 缓存应用中的低效性。最后, 我们探讨了克服这些低效性的潜在设计方向, 并强调了实现此类系统的关键挑战。

2.1 大语言模型推理与键值缓存

预填充和解码。现代大模型基于 Transformer 架构 [48]。token 生成包含两个阶段: 预填充和解码。在预填充阶段, 模型并行处理输入提示, 将 token 转换为向量, 并计算查询 (Q)、键 (K) 和值 (V) 矩阵。预填充通常为计算密集型, 以首个 token 生成时间 (TTFT) 衡量, 即处理完整输入并输出第一个 token 所需的时间。在解码阶段, 模型根据先前生成的 token 自回归地逐个生成 token。通常使用 token 间延迟 (ITL) 来描述解码性能。

KV Cache: Trading Memory for Compute. 为避免在解码阶段重复计算 token, 推理引擎使用键值 (KV) 缓存来存储先前已计算的 token。预填充 (prefill) 阶段生成的 K 和 V 矩阵被保存下来, 并在后续的解码步骤中复用。KV 缓存并非与会话绑定: 对于共享相同提示 (prompt) 的请求, 该缓存可跨请求复用, 这一技术称为前缀缓存 (prefix caching)。当提示命中缓存时, 可跳过预填充阶段, 从而释放计算容量, 并将每个 token 的成本最多降低 ~90% [7, 36]。这有助于 GPU 更快地生成 token, 维持更高的每秒查询数 (QPS), 进而提升服务等级目标 (SLO) 和用户体验。

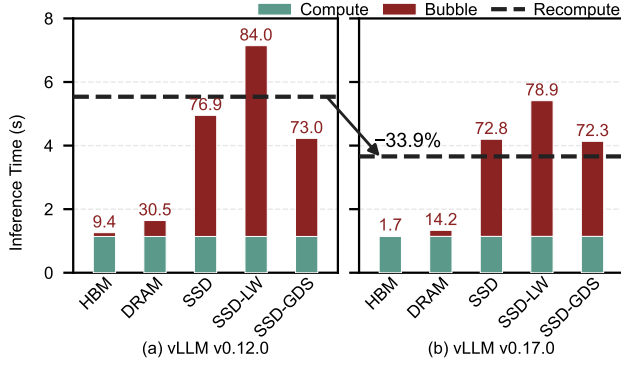


图 2. vLLM 结合 LMCache 在 Llama3-8B 上的推理性能，涵盖 HBM、DRAM 和 SSD 层级（序列长度 = 64K，命中率 = 75%）。DRAM 性能接近 HBM，而 SSD 与 GDS 则导致显著的 GPU 空闲等待。虚线标记了重新计算的性能表现。随着大模型引擎持续优化推理计算，从 SSD 恢复 KV 缓存已不再具有优势（vLLM v0.12.0 vs. v0.17.0），原因在于严重的 I/O 瓶颈。

Paged KV Memory Management. KV 缓存的内存占用随输入长度增长，而变长请求会导致 GPU 内存碎片化。为解决该问题，现代推理系统 [19] 沿层维度和 token 维度将 KV 缓存划分为形状为 $[Block, h, d]$ 的非连续块，每个块通常容纳 16–32 个 token。这些块按需分配，以支持动态序列增长，并与逐层计算对齐。这种分页式布局已成为现代大语言模型（LLM）推理引擎（例如 vLLM [20]、SGLang [42] 和 TensorRT-LLM [32]）的事实标准。

2.2 分层键值缓存中的 SSD 引起的瓶颈

随着上下文窗口扩展到数百万 token [29, 47] 且活跃会话数量增加，总的 KV 缓存占用迅速超出 GPU HBM 容量 [27]。KV 缓存卸载通过 CPU DRAM 和 NVMe SSD 扩展了 GPU HBM 容量，形成了两级的 HBM-DRAM 和三级的 HBM-DRAM-SSD 层次结构。HBM-DRAM 层次结构仅带来轻微的性能下降，但扩展容量有限。相比之下，HBM-DRAM-SSD 层次结构提供了更高的容量，但带来了显著的 I/O 开销。

当将 KV 缓存卸载到 SSD 时，主要挑战源于分页 KV 布局与 SSD 访问模式之间的不匹配。一旦非连续的 GPU KV 块被逐出到 SSD，内存碎片化 [43] 就会演变为严重的 I/O 碎片化。对于一个 64 层的 Qwen3-32B 模型 [53]（块大小为 64），重新加载一个 128K token 的 KV 需要获取约 256 K ($= 2 \times 64 \times 128 \times 1024 / 64$) 个分散的 80 KB 对象。这种访问模式会产生大量小规模、随机的传输操作，导致 CPU-GPU 数据拷贝、文件系统以及 I/O 提交开销 [23, 39, 40, 43, 50] 成为数据移动的主要瓶颈。将多个块组合成更大的数据块虽可提升 I/O 效率，但会在传输效率、前缀共享效果和缓存管理粒度之间引入权衡。例如，默认的 LMCache [26] 数据块存储 256 个 token，使得一个 128K token 的 KV 需要超过 1,000 次数据块访问，其中绝大多数为随机访问。在计算-I/O 流水线机制下，访问次数进一步增至数万次。结果，大量昂贵的 GPU 计算

周期被浪费在等待从 SSD 恢复 KV 缓存上，导致 KV 缓存重用速度甚至比重新计算还慢。

SSD Tiers Cause Growing GPU Bubbles. 为了考察这些瓶颈在实际中的表现，我们采用最新版本（v0.4.2）的 LMCache [12, 26] 作为分层 KV 缓存存储的代表。LMCache 支持 DRAM 和 SSD 层级、逐层计算-输入输出流水线 [13, 16]，以及（可选的）GPU 直接存储（GDS）[33]。我们在不同 vLLM 版本（v0.12.0 于 2025 年 12 月发布，v0.17.0 于 2026 年 3 月发布）下评估 Llama3-8B 模型，序列长度为 64K，命中率为 75%，DRAM-HBM 带宽为 50 GB/s，使用两块 SSD，读取峰值带宽为 29 GB/s，写入峰值带宽为 12 GB/s（详见 §4 的详细配置）。

① *DRAM tier is efficient.* 如图 2 所示，从 CPU DRAM 加载 KV 相对于 HBM 仅引入了适度的开销。低延迟、细粒度的 DRAM-HBM 访问，结合 LMCache 的 GPU 辅助拷贝，将大量顺序的 `cudaMemcpyAsync` 调用合并为少量具有最小控制开销的 GPU 内核。此外，DRAM 的低延迟和强大的随机访问性能使得逐层流水线能够有效隐藏数据移动带来的开销，使其在注意力计算中得以隐藏。

② *SSD 层即使使用 GDS 也效率低下。* 当将层次结构扩展到 SSD 时，即使采用聚合 KV 传输和异步 I/O [10]，恢复 KV 缓存仍然非常低效。如图 2 所示，从 SSD 恢复 KV 缓存的性能远低于从 DRAM 恢复，导致在所有情况下 GPU 空闲时间均超过总推理延迟的 70%。在 SSD 上应用逐层传输（SSD-LW）进一步降低了 I/O 粒度并增加了操作次数，加剧了端到端延迟，使 GPU 空闲时间达到总推理延迟的约 80%。

GDS [33] 通过点对点 DMA 消除了 CPU-GPU 数据拷贝，但仍依赖 CPU 干预来启动每次 I/O 操作，导致显著的软件开销，并限制了 I/O 并行性 [8, 23]。即使使用 GDS，GPU 空闲时间仍高达 70% 以上，表明仅将 CPU 从数据路径中移除几乎无法缓解分页 KV 布局与 SSD 访问模式之间的不匹配问题。此外，随着 LLM 引擎持续优化推理计算，由于严重的 I/O 瓶颈，从 SSD 恢复 KV 缓存已不再具有优势。

2.3 以 GPU 为中心的存储

GPU-centric storage [5, 28, 39, 40] 将数据平面和 I/O 控制平面均迁移至 GPU。它使得 GPU 线程能够无需 CPU 干预即可发起 NVMe I/O。BaM [40] 首次在 GPU 内存中直接管理 NVMe 提交队列（SQ）和完成队列（CQ），从而使 GPU 内核能够从设备代码中完全自主地入队 I/O 命令、触发 NVMe 门铃并观察完成状态。这降低了 CPU-GPU 同步开销和内核启动开销，使大量并行的 GPU 线程能够驱动高带宽、细粒度的 I/O。

Common GPU-centric storage abstraction. 在以 GPU 为中心的存储系统中，GPU 线程通过块或文件接口与高通量软件缓存（例如 BaM 中的数组或 GeminiFS [39] 中的页缓存）进行交互。当发生缓存未命中时，GPU 线程将 I/O 请求放入 GPU 地址空间中的 NVMe 提交队列，并触发门铃寄存器。随后，它会轮询完成队列，直到数据到达。通过交错不同线程束的 I/O 和计算阶段，这种以

GPU 为中心的设计能够重叠计算与存储访问，并隐藏延迟。

Implications for KV cache workloads. 尽管以 GPU 为中心的存储提供了一个有前景的方向——由 GPU 控制、细粒度地访问 NVMe——但其设计基于通用的块和文件抽象，并在线程或线程束级别保持忙等待。如我们接下来所示，这种抽象与大语言模型推理中的键值缓存布局及紧密流水线化的解码不相匹配，导致诸如控制开销过大、请求合并效果差以及在朴素应用于分层键值存储时 NVMe 带宽利用率低下等问题。

2.4 GPU 中心存储在键值缓存中的挑战

将面向 GPU 的存储应用于键值缓存工作负载，在抽象、粒度和竞争方面面临独特的挑战。

Abstraction mismatch for KV cache management. 大语言模型引擎（vLLM [20] 和 SGLang [42]）需要为 KV 缓存动态分配和索引 GPU 内存块，而以 GPU 为中心的存储仅提供底层磁盘块和文件接口。将这一管理下放到 GPU 上，需要在设备代码中实现基于哈希的分配和查找。然而，如图 3 所示，GPU 上的哈希表性能较差：在不同序列长度下，插入和查找开销分别比 CPU 哈希表高出 $9.0\times \sim 24.2\times$ 和 $25.6\times \sim 50.0\times$ ，每次操作耗时可达数秒。这很难解决，因为哈希计算和探测形成了串行依赖链，且每个块的哈希值依赖于前一个块。这种不规则、指针追踪的工作负载难以映射到 SIMT 执行模型，无法有效利用 GPU 并行性。

Granularity gap between storage I/O and KV transfers. GPU NVMe 驱动针对细粒度、类似缓存的访问进行了优化，但 KV 缓存重载需要中等大小、连续的数据传输以满足 SSD 带宽目标。在 PCIe 5.0 SSD 上 [18, 46]，4KB 的请求可以饱和 IOPS，但仅使用约 80% 的读取带宽和 16% 的写入带宽，导致可用吞吐量显著未被充分利用。简单地增大请求大小并非易事。GPU NVMe 驱动依赖于 NVMe 物理区域页（PRPs）来向控制器描述 GPU HBM 地址。固定的 4KB PRP 可由 CPU 驱动预先分配，但 KV 缓存传输的大小是可变的且大得多（ ~ 100 KB）。对于超过 8KB 的请求，NVMe 需要额外的 PRP 列表页，其分配和地址转换必须在特权 CPU 代码中完成 [39, 49]。由于 GPU 程序以非特权模式运行，以 GPU 为中心的存储难以轻松粗化 I/O 而不回退至 CPU，这会削弱消除 CPU 干预的目标。

Resource contention. GPU-centric storage I/O competes with LLM computation for resources.

① **SM 竞争。** 大语言模型推理具有严格的依赖关系：注意力机制必须等待相应的键值缓存可用后才能进行。现有的以 GPU 为中心的存储设计采用同步的忙等待 I/O，即 GPU 线程在计算内核内部持续轮询完成队列并阻塞计算。若未进行仔细解耦，单纯增加更多的 I/O 并行性只会减少可用于计算的 SM 预算。

② **带宽竞争。** 前缀缓存会产生大量双向流量。特别是在计算-输入/输出流水线阶段，来自前一层的并发写入操作和针对下一层的读取操作会导致对 NVMe 资源（例如，SSD 内部缓存）的竞争，从而降低 NVMe 带宽

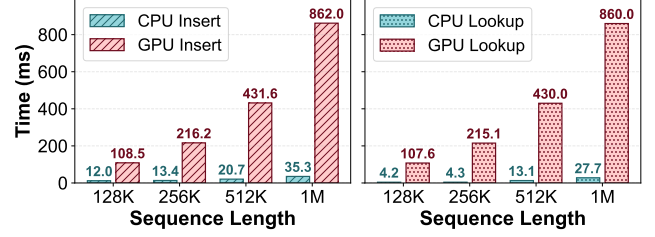


图 3. CPU 与 GPU 在哈希性能方面的对比。

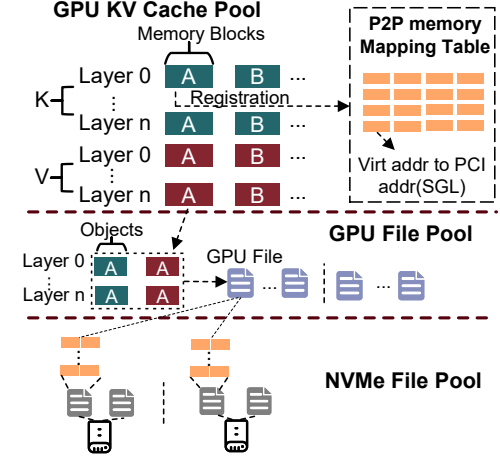


图 4. 以 GPU 为中心的键值缓存存储布局。

(§3.3)。在 GPU 内部实现细粒度的 I/O 调度以提高利用率非常困难，而分离调度则往往会导致内核执行时间增加。

3 设计与实现

在本节中，我们介绍 *Tutti* 的设计与实现。我们首先描述了支持高并发 GPU 直接访问 KV 缓存的原生 GPU 对象抽象，该抽象采用对象语义。接着，我们解释了应用程序如何高效地提交并获取异步 GPU I/O 内核的结果。最后，我们讨论了如何调度 GPU I/O 内核以最小化资源竞争。

3.1 以 GPU 为中心的对象存储

Tutti 的核心是一个以 GPU 为中心的 KV 缓存对象存储。如 §2.4 所讨论，KV 缓存管理无法完全交由 GPU 处理：索引、跨请求的全局共享以及对引擎可见的映射仍需与 CPU 端的推理引擎保持协调。

幸运的是，所有管理逻辑均可由 CPU 在 KV 缓存传输的关键数据和 I/O 控制路径之外（off）处理。因此，我们在 GeminiFS [39] 之上构建了以 GPU 为中心的对象存储系统；GeminiFS 是一种面向 GPU 的配套文件系统，它与传统的 CPU 端文件系统（例如 ext4）共存，从而使得文件系统元数据可在 CPU 上进行管理，并与 GPU 共享。我们对 GeminiFS 进行了扩展，引入了一个可扩展的 GPU 文件池以及一个用于动态批量 KV 缓存传输和 KV 管理操作（例如 Store 和 Retrieve）的 P2P 内存映射表。

Scalable GPU File Pool. 如图 4 所示, Tutti 通过将每个内存块表示为一个对象, 使存储分配与推理引擎的 KV 块管理器对齐。一个 GPU 文件被组织为 $2 \times L$ 个对象 (其中 L 为层数), 每一层对应一个键对象和一个值对象。该映射保留了推理引擎原生的块粒度, 使得动态分配、索引和共享在 HBM 和 SSD 两级存储中保持一致。

GPU 文件对推理引擎可见, 而 NVMe 文件由 GeminiFS 管理, 作为分配给 SSD 的物理存储扩展。Tutti 使用 *Tensor-Stripe* 布局将每个 GPU 文件映射到多个 NVMe 文件, 该布局遵循原始张量粒度, 而非细粒度的存储分条。因此, GPU 文件的形状与 KV 缓存内存对象 ($2 \times \text{layer} \times \text{block}$) 匹配, 使得存储 I/O 保持与 KV 传输粒度对齐。对于跨越多个 GPU 文件的提示, 我们采用设备间轮询放置策略。具体而言, 对象以行顺序方式均匀分布在多个 NVMe SSD 上。这种方法不仅平衡了各驱动器之间的 I/O 流量, 有助于饱和聚合的 NVMe 带宽, 还减少了 GPU 文件与 NVMe 文件之间的索引开销。

系统启动时, Tutti 会在每个设备上预先分配大量 NVMe 文件, 并将其暴露为可用的 GPU 文件。当需要持久化新的 KV 缓存时, 运行时仅需选择一个空的 GPU 文件, 并在 CPU 端建立从 KV 缓存到 GPU 文件 ID 的哈希映射。这在保留运行时所期望的动态分配语义的同时, 将文件创建、回收及其他元数据操作从运行时的关键路径中移除。

P2P Memory Mapping Table. GPU 文件池解决了逻辑对象管理问题, 而剩下的挑战是在运行时 I/O 提交过程中将 KV 缓存的虚拟地址转换为 PCI 可见的物理地址。由于现代推理引擎在初始化时预分配一个固定的 KV 缓存内存池, 并在整个生命周期内保持稳定, 因此 Tutti 可以在启动时预先计算出一个 P2P 内存映射表, 并在后续的 GPU I/O 中重复使用。

然而, 直接基于 PRP 的设计会导致显著的内存开销。例如, 对于 80 GB HBM 上的 60 GB KV 缓存, PRP 需要为每个页都分配一个指针 ($\text{Total Pages} = 60 \times 1024^3 / 4096 = 15,728,640 \text{ Pages}$)。如果以 64KB 粒度分配 PRP 列表页 (每页仅存储 16 个指针), 则需要 983,040 页。这导致实际 HBM 使用量达到 ($983,040 \times 4 \text{ KB} \approx$) 3.75 GB), 显著浪费了昂贵的 HBM 资源。

为了更好地匹配中等大小的 KV 传输, Tutti 采用分散收集列表 (SGL) [49] 而非 PRP。它仅使用 16 字节描述一大块连续内存, 包含物理地址 (8 字节)、长度 (4 字节) 和标识符 (4 字节)。因此, 内存消耗降低至 ($983,040 \times 16 \text{ B} \approx$) 15 MB。

运行时, 推理引擎仅执行块查找和点对点表查找, 以生成一批轻量级的 GPU I/O 上下文, 随后将这些上下文传递给 GPU 进行并发执行。这避免了在关键 I/O 路径上为每个请求构建物理地址和文件管理的开销。引擎可见的映射关系仍由 CPU 管理, 而 GPU 仅持有用于直接 I/O 提交所需的元数据。因此, Tutti 提供了逐层批量的 Store 和 Retrieval 接口, 将 CPU 开销从 $O(\text{layer} \times \text{blocks})$ 降低至 $O(\text{layer})$ 。

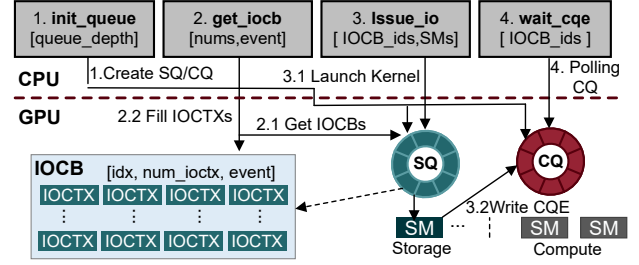


图 5. GPU io_uring 的架构与 I/O 流程

3.2 GPU io_uring

GPU 中心的对象存储遵循“CPU 准备, GPU 执行”的模型。CPU 运行时从 CPU 管理的映射中准备 I/O 控制块 (IOCBs), 并提前将 GPU I/O 内核与模型计算内核一并入队。一旦入队, GPU 端的依赖跟踪将决定何时发起 SSD 访问, 因此运行时 I/O 关键路径不再涉及 CPU。这自然实现了 GPU I/O 与计算内核的解耦, 同时支持 I/O 内核与计算内核之间的高效并行。由于其设计与 CPU 端的 io_uring [10] 类似, 我们称之为 GPU io_uring (gio_uring), 其架构如图 5 所示。

Zero-Copy Ring Buffers. 为避免在 CPU 提前准备 GPU I/O 工作时产生运行时内存分配、复制以及 CPU-GPU 同步开销, gio_uring 利用一对位于 GPU HBM 中但通过非缓存 mmap 映射到 CPU 的无锁环形缓冲区 (SQ 和 CQ)。[52] 为了支持数千个并发的 GPU I/O 请求, 该系统采用批量队列结构。与传统的 CPU io_uring (其中每个 SQ 条目对应一个单一命令) 不同, 每个 SQ 条目被定义为一个 I/O IOCB, 每个 IOCB 包含 2048 个 I/O 上下文 (IOCTXs)。

IOCTX 记录了 SGL 地址、GPU 文件偏移量和长度。IOCTX 的数量与 GPU 的极小调度单元对齐。例如, 在 H100 上 (其单元为 2 个 SM), 每个 SM 支持 64 个线程束, 每个线程束包含 32 个线程, 总共可支持 4096 个并发线程。考虑到寄存器压力, 我们通常将理论上限除以 2。这种设计使 GPU 能一次性提交大量 I/O 请求。

SM Partitioning For Accurate I/O. 使用多个 CUDA 流实现的简单并发无法实现计算与 I/O 之间的细粒度重叠。由于 GPU 硬件调度器具有高度非抢占性 [24], 一个长时间运行的 I/O 内核会独占资源, 即使有空闲的 SM, 也会阻塞另一个关键计算内核的执行。通过 NVIDIA 绿色上下文 [34], 我们在硬件层面将 GPU 资源隔离为“计算域”和“I/O 控制域”。I/O 控制内核在专用 SM 上运行, 不受计算负载波动的影响。这确保了对延迟敏感的内核能够尽快启动并完成。该设计避免了传统协作多任务中常见的长尾延迟和资源饥饿问题, 并提供了确定性的服务质量 (QoS)。

Async I/O Processing: gio_ring 的处理过程与传统的 CPU 侧 io_uring 类似: ① *init_queue(depth)* 创建一个包含 depth 个 IOCB 的提交队列 (SQ) 和完成队列 (CQ), 每个 IOCB 拥有唯一的索引。② *get_ioctx(nums, event)* 在执行前被调用以获取所需的 IOCB。应用程序将这些 IOCB 填入 CPU 侧的虚拟地址, 并更新 *num_ioctx*。为保证在乱序流执行

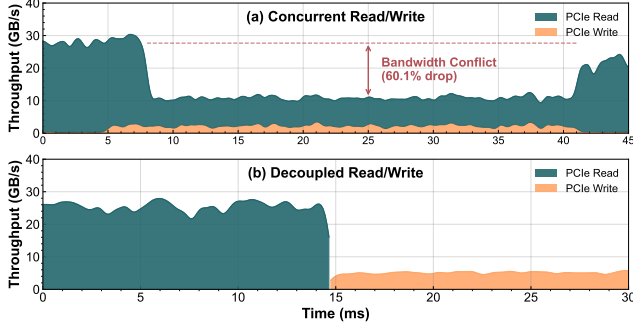


图 6. 并发与解耦读写 PCIe 带宽利用率。

下的正确性，插入一个 CUDA 事件，确保 GPU I/O 内核仅在满足所需依赖后才开始执行。③ `issue_io(IOCB_ids, SMs)` 使用指定的 IOCB ID 和 SM 分配将 GPU I/O 内核入队，实现设备内并行。内核入队后，全部在 GPU 上生成并发出 SSD 命令。当内核执行完毕时，原子地将 IOCB 索引写入完成队列 (CQ)。④ `wait_cqe()` 通过检查完成队列中特定的 IOCB 索引来提供细粒度等待，无需 CPU 参与每次 I/O 的发起。

3.3 感知延迟的 I/O 调度器

仅使用异步 I/O 和 SM 划分不足以实现稳定的计算-I/O 重叠，因为仍存在两种干扰源。首先，读取和写入 I/O 会竞争 SSD 带宽和内部资源。这在简单的逐层流水线中很常见，其中新生成的 KV 的写入 I/O 会与下一层次的读取 I/O 竞争。这种损失并非简单的叠加共享效应：如图 6 所示，在读写并发时总带宽下降了 60%，而分离调用则可使设备达到饱和。这主要是因为大块读取和写入会竞争 NVMe 的内部缓存 [15, 25]，我们通过 FIO 使用一个读取线程和一个写入线程、以 256 MB 为粒度重现了这一行为。

其次，I/O 内核也会与模型执行竞争 SM 资源。嵌入、规范化和 GEMM 等操作可能占用高达 90% 的 GPU 资源。在非抢占式 GPU 调度下，因此长时间运行的 I/O 内核可能会延迟关键的计算内核，从而降低推理性能。

为应对这两种影响，Tutti 提出了一种基于查找表的、考虑空闲时间的 I/O 调度器，如图 7 所示。空闲时间指的是具有剩余 SM 资源且无有害读写带宽竞争的执行窗口。该调度器利用离线分析结果，仅将读和写内核放置于这些窗口中，从而最小化对模型执行的干扰。

Offline Profiling of SM Slack Windows. 预填充 (Prefill) 复杂度随前缀长度变化。变化的主要来源是注意力 (Attention) 复杂度。随着前缀长度增加，每个新 token 所需的注意力操作数呈线性增长，导致其浮点运算量 (FLOPs) 高于零前缀基准 (zero-prefix baseline)。相反，层内其他算子 (例如线性的投影 (Linear Projections) 和规范化 (Normalization)) 不受上下文长度影响。因此，我们对每一层进行离线性能分析，并将所得松弛 (slack) 信息存入一张查找表 (lookup table)，该表以输入长度 (L_{input}) 和前缀长度 (L_{prefix}) 为索引。每条表项记录可调度松弛窗

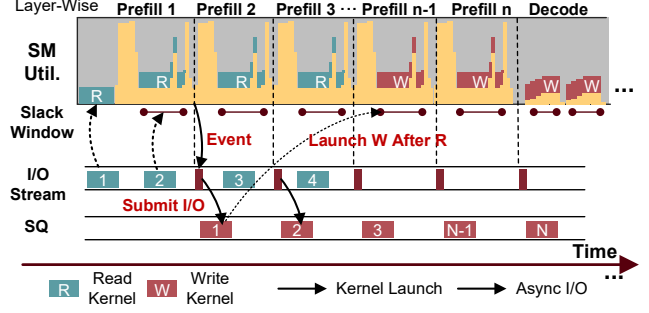


图 7. 感知延迟的 I/O 调度器。

口 (schedulable slack windows) 的持续时间及可用流式多处理器 (SM) 预算，使运行时系统能直接通过查表确定在不进行在线建模的前提下最多可启动多少个 IOCB。步长 (step size) 与单个线程束 (warp) 所处理的 token 长度对齐，从而大幅减少离线性能分析所需时间及数据规模。此外，我们还针对不同 IOCB 数量分别分析解码 (decode) 耗时，以及读/写内核 (read/write kernels) 的执行时间和 SM 占用率，从而使调度器可通过查表选择合适的启动规模。

Decoupled Scheduling for Read and Write. 为避免因并发读/写执行导致的带宽崩溃，Tutti 并不采用简单粗暴的逐层流水线方式 (即不加区分地重叠读操作与写操作)。相反，它根据已分析得到的松弛时间查找表 (slack table) 对读操作与写操作进行分离调度。在预填充 (prefill) 阶段，读内核具有更高优先级，因为 KV 检索位于重用的关键路径上。当推理请求到达时，运行时系统将对读操作的 IOCB (Input/Output Control Block) 入队。在每一层开始之前，调度器依据当前输入长度与前缀长度查询该查找表，并在下一个已分析的松弛时间窗口 (profiled slack window) 内启动尽可能多的 IOCB。若不存在合适的松弛时间窗口，则表明 KV 检索已成为高重用场景下的瓶颈，此时调度器将立即启动所需读操作，以避免计算停滞。

写请求仅在关键路径读取被调度后才被处理。待处理的写操作仍记录在 SQ 中，`gio_uring` 会自动插入 CUDA 事件以保证正确性。如果当前预填充层仍存在可调度的空闲时间窗口，调度器将根据查找表允许的数量发出尽可能多的写操作；否则，将其推迟以缩短预填充时间并保持 TTFT。剩余的写操作将在解码阶段通过尽力而为的策略进行刷新。尽管解码阶段通常提供的 GPU 利用率较低，但其空闲时间窗口较短且不可预测，因此调度器依赖查找表来机会性地发出写操作。未能及时处理的请求将保留在队列中，等待后续的空闲时间窗口，从而减少请求间的干扰并提升吞吐量。

3.4 Tutti 实现

Integration with vLLM. We implemented Tutti using ~8,000 LoC in C++ and integrated it with vLLM's KVConnector in multiple versions using ~1,500 LoC in Python. This integration preserves vLLM's block-granular KV management.

The GPU file pool exposes layer-wise `retrieve_layer` and `store_layer` interfaces to support efficient layer-wise KV movement in vLLM. This organization matches the layer-wise transfer model described earlier and creates opportunities to overlap KV movement with model computation. The extension to vLLM is used to register the pre-allocated KV memory block pool, identify reusable prefixes, and construct the mapping from logical KV blocks to GPU files.

`Retrieve_layer` 在重用的关键路径上发出, 而 `store_layer` 则被排队并根据需要延迟, 以便在后续的空闲时间段内刷新, 包括后续的请求, 从而减少请求间的干扰。为保持正确性并限制 GPU 资源的使用, 这些接口与 CUDA 流依赖关系绑定, 而详细的 GPU 侧提交和完成流程遵循第 3.2 节。调度决策随后由感知空闲时间的调度器负责。

在预热阶段, Tutti 会为给定模型和系统配置分析每一层的松弛窗口。生成的性能分析结果只需创建一次, 即可在相同部署情景下的多次推理过程中重复使用。在每次调用 `retrieve_layer` 或 `store_layer` 之前, 运行时会查询当前层的松弛条目, 以决定是否发起 I/O 操作以及启动多少个 IOCB, 从而最大限度减少与推理内核的资源竞争。

Support for Multi-GPUs. 当模型采用张量并行等多 GPU 部署方式时, vLLM 为每块 GPU 启动一个进程, 从而可在每个 GPU 进程旁部署一个 Tutti 实例。每个 Tutti 仅管理部分 KV 缓存; 各进程独立负责其 GPU 上驻留层所对应的 KV 块, GPU 文件大小亦随之相应调整。为支持 GPU 间共享 NVMe, 我们采用一个本地守护进程, 该进程分配 GPU 内存, 并为每块 GPU 初始化一对专用的 NVMe 提交/完成队列。对应的 vLLM 进程通过 GPU 进程间共享内存获取其 GPU 驻留队列的地址, 并直接经由这些队列提交 I/O 命令。由于每块 GPU 拥有独立的队列对, 因此不存在 GPU 间的队列争用问题, 使得所有 GPU 均可并行访问本地 NVMe, 实现高通量的 KV 读写。我们在原型中使用的 Solidigm D7-PS1010 [45] 支持最多 256 个 I/O 队列, 因而可为 8 块 GPU 中的每一块分配 32 个队列。该队列数量已足以使 Tutti 充分利用单块 SSD 的带宽。

Scalability. 为了超越单个结点的限制, Tutti 将其本地高性能存储数据平面与分布式协调层相结合。在此设计中, Tutti 仍保持为 GPU 到本地 NVMe KV 传输的每服务器快速路径, 而 Mooncake [38] 则作为跨簇的控制平面, 负责空间分配、副本元数据管理以及位置查找。这种分离保留了 Tutti 的低延迟本地路径, 同时允许 KV 缓存容量和复用在推理服务器之间扩展。

当 KV 缓存从 GPU 内存中被逐出时, 推理引擎首先向 Mooncake 请求空间分配。Tutti 随后通过其 P2P DMA 路径将 KV 张量持久化到本地 NVMe SSD 上。写入完成后, 它会通知 Mooncake 注册生成的副本元数据, 从而使卸载的 KV 对未来的重用全局可发现。

当请求需要重用历史 KV 缓存时, 运行时首先向 Mooncake 查询候选副本位置。系统采用本地优先的路由策略。如果存在本地副本, Tutti 会直接通过 Tutti 将其加载到

GPU 内存中。否则, 请求将回退到远程获取路径, 从远程结点获取数据, 然后传输到本地 GPU。

我们的当前原型尚未优化此远程路径。它使用 CPU 端接口将 GPU 文件读取到主机内存, 然后通过 RDMA 在结点间传输, 这虽然最小化了对 Mooncake 的修改, 但增加了额外的 CPU 开销。在后续工作中, 我们计划扩展设计以支持更直接的 GPU 驱动远程路径, 例如通过将数据暂存于 GPU 内存, 然后由 GPU 发起 RDMA 操作至目标 GPU。

4 评估

我们进行了一系列实验, 以评估 Tutti 的有效性, 重点关注以下两个关键问题:

1. Tutti 在大语言模型推理的端到端延迟方面, 相较于当前最先进的 KV 缓存服务表现如何?
2. 图蒂 (Tutti) 的各个组件如何促进并优化最终推理延迟和整体系统效率?

环境. 我们将 Tutti 部署在一台配备 512 GB 内存的 64 核 Intel Xeon 6530 服务器上。该服务器配备了两个具有 80GB HBM 的 H100 GPU 以及 4x Solidigm D7-PS1010 7.68TB 企业级 SSD [45]。对于分层存储配置, 我们为每个 GPU 分配 256 GB 主机 DRAM 作为固定内存, 并配置 14 TB 的 SSD 存储空间。

基准. 我们将 Tutti 与两个版本的 vLLM 基准进行对比: vLLM 0.12.0 和 vLLM 0.17.0。该设置使我们能够考察服务端计算效率的提升如何影响端到端系统行为。LMCache 通过将 token 聚合为粗粒度块 (例如, 256 个 token) 来优化数据移动, 以最大化 SSD 带宽, 这与 vLLM 采用细粒度 64-token 分页形成对比。为了评估在不同层级存储系统上的性能表现, 我们配置了以下四种基准: (1) HBM: 仅使用 HBM 的标准 vLLM 服务; (2) DRAM (LMCache-DRAM-LW): 利用主机内存扩展容量, 并应用逐层计算-I/O 流水线以重叠检索开销; (3) LMCache-SSD: 使用 memcopy 和标准异步 I/O 将 KV 数据卸载至 NVMe SSD; 以及 (4) LMCache-GDS: 进一步利用 GDS 优化 SSD 访问, 绕过 CPU 缓冲区。除非在端到端结果中另有说明, 否则 DRAM 指 LMCache-DRAM-LW; 在消融实验中, 我们额外报告不使用逐层传输的 LMCache-DRAM。

模型. 我们主要在单个 GPU 上使用 Llama3-8B[30] 模型评估性能。为了评估我们的系统在超长序列推理中的可扩展性, 我们额外采用了 GLM-4-9B-Chat-1M[14]。该模型支持 100 万 token 的上下文窗口, 通过张量并行分布在两个 GPU 上。

工作负载. 我们使用两个已建立的基准: LEval[2] 和 LooGLE[22]。

LEval 是一个全面的长上下文评估套件, 包含 20 个子任务, 分为两大类, 覆盖法律、金融、技术、学术论文和代码等多个领域。LEval 中的输入长度跨度广泛, 从 3k 到 200k token 不等。LooGLE 包含 4 个子任务, 专为超长上下文理解设计, 其文档平均长度显著更高, 许多测试样本超过 100k token。它聚焦于长依赖问答和单轮摘要等复杂任务。

在当前的系统配置下, 各存储层级的缓存命中率如表 1 所示。HBM 容量不足以支持长上下文服务, 在 LEval

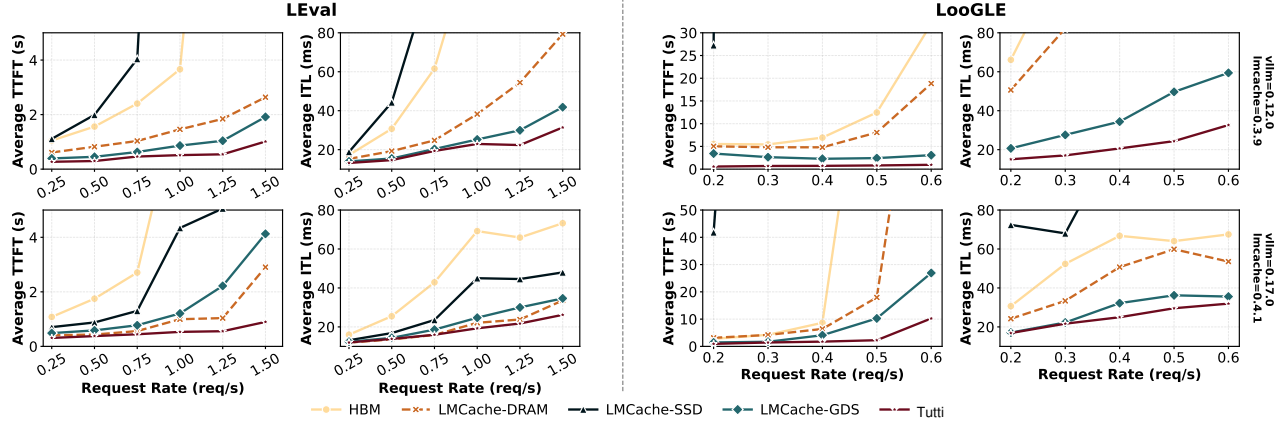


图 8. 在最新版 LMCACHE 下，针对 Llama3-8B 模型，在 LEval 和 LooGLE 两个基准上，对比 vLLM 两个版本（v0.12.0 与 v0.17.0）的端到端 TTFT 和 ITL。随着请求速率上升，Tutti 始终保持最低且最稳定的延迟曲线，这与端到端分析结果一致，表明其存储-计算协同设计在不同版本间均保持高效。当系统违反 SLO 约束时，相应数据点被省略。

表 1. 不同存储层级的缓存命中率。

Storage Medium	Cache Hit Rate (%)	
	LEval	LooGLE
HBM	8	4
DRAM	53	24
SSD	84	86

和 LooGLE 上的命中率分别仅为 8% 和 4%。DRAM 将重用率提升至 53% (LEval) 和 24% (LooGLE)，但由于 LooGLE 中上下文长度较大，仍存在大量未命中。相比之下，SSD 在两个场景中均保持了稳定的高命中率（分别为 84% 和 86%），表明大部分可重用的 KV 状态均可被大容量的 SSD 层捕获。

为了模拟多会话环境，我们采用轮询策略从 LEval 和 LooGLE 的各个子数据集中提取请求。为了评估系统在不同负载条件下的鲁棒性，我们通过泊松分布模拟查询到达情况，因为这些数据集缺乏原生时间戳。此设置与先前研究中采用的评估协议一致 [20, 38]。这些请求被持续推送至 vLLM 服务引擎，模拟真实场景中多个用户同时提交具有不同上下文长度的多样化查询。

指标。 我们使用两类指标对 Tutti 进行评估：端到端应用性能和系统级微基准测试。我们重点关注两个标准的推理延迟：(1) TTFT，用于衡量预填充阶段的响应速度；以及 (2) ITL，用于量化解码速度。我们报告在并发负载下的平均延迟。

为剖析系统各组件的贡献，我们进行如下测量：(1) 缓存命中率，重点分析其对降低首字延迟 (TTFT) 的影响；(2) 存储带宽，用于评估存储引擎的原始吞吐能力；(3) GPU 空闲时间，用于评估异步 I/O 调度在隐藏延迟方面的有效性；(4) 推理成本，用于评估设计的成本效益。

4.1 端到端性能

如图 8 所示，与所有基准方法相比，Tutti 在端到端性能和稳定性方面表现更优。在更新的软件版本（vLLM 0.17.0）下，Tutti 在两种工作负载上仍能提供最佳的端到端延迟，证实了即使服务引擎变得更加计算高效，我们的存储-计算协同设计依然有效。

首次 token 延迟。 在两个软件版本中，HBM 和 SSD 基准在 TTFT 方面表现均较弱。HBM 受限于容量有限和命中率低，导致频繁的 KV 重新计算。SSD 受限于较长的 I/O 延迟以及 CPU 端软件开销（例如内存分配/释放），增加了 I/O 震荡和排队延迟，进而延长了 GPU 等待时间。在旧版本的 LEval 测试中，DRAM、GDS 和 Tutti 在高请求速率 (RPS，每秒请求数) 下均能提供可用的 TTFT，其中 Tutti 表现最佳，在最高负载点相比 GDS 提升了 71.8%。在新版本中，计算速度提升，使得 GDS 的 I/O 路径相对开销更加明显；在高负载下，DRAM 相比 GDS 将 TTFT 降低了 29.6%。即便在此变化下，Tutti 仍保持最优，相比 DRAM 减少 TTFT 69.1%，相比 GDS 减少 78.3%。在 1 秒 TTFT SLO 条件下，Tutti 相比 DRAM 将有效请求速率提升了 50%，相比 GDS 提升了 100%。在 LooGLE 测试中，由于请求更长，两个版本中 HBM 和 SSD 均持续表现较差。在旧版本中，GDS 仍显著优于 DRAM，且与 Tutti 的差距相对较小。在新版本中，GDS 继续优于 DRAM，但其相对优势下降，在 0.6 RPS 时其 TTFT 仍约为 Tutti 的 2.63×。在同一负载点，Tutti 相比 DRAM 将 TTFT 降低 93.2%，相比 GDS 降低 62.0%。

Token 间延迟。 在旧版本的 LEval 上，Tutti 在高负载下已优于 DRAM 和 GDS：在 1.5 RPS 时，ITL 相比 DRAM 降低 60.4%，相比 GDS 降低 24.9%。在新版本中，Tutti 依然是最优的解码路径；在 LEval 上 1.5 RPS 时，ITL 仍相比 DRAM 降低 22.0%，相比 GDS 降低 24.4%。这一提升来自两个方面：Tutti 在解码过程中提供了更高的有效缓存命中率，并缩小了计算与 I/O 之间的差距，使得 GPU 能够将更多周期用于生成有效 token，而非等待数据。在

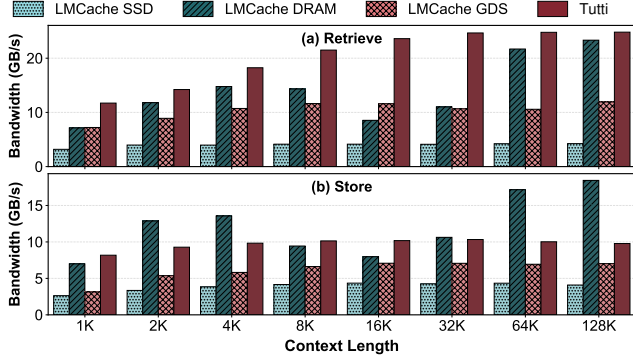


图 9. 不同上下文长度下，检索和存储接口的原始带宽。

LooGLE 上，新版本的收益有所收窄（在 0.5 RPS 时相比 GDS 降低 18.3%，在 0.6 RPS 时降低 10.2%），但 Tutti 依然保持持续领先。在 LooGLE 上差距收窄的原因是更长的输入导致每个 token 的计算时间增加，使解码相对更偏向于计算密集型。即便如此，Tutti 仍保持最低且最平滑的 ITL 曲线，表明在相同 ITL 目标下仍有潜力维持更高的 RPS。

4.2 消融

在本小节中，我们进行消融实验以分离 Tutti 中关键设计组件的贡献。我们评估了五个方面：原始检索/存储带宽、PRP 与 SGL 命令路径、不同前缀复用情况下的 TTFT、分布式可扩展性，以及逐层异步流水线的有效性。这些消融实验直接评估了我们以 GPU 为中心的对象存储路径中的关键要素，包括命令提交开销、传输带宽和重叠效率。为使本节中的 DRAM 基准更加明确，我们额外报告了 LMCached-DRAM 和 LMCached-DRAM-LW 两种配置。LMCached-DRAM 表示不采用逐层（LW）拷贝/重叠的 DRAM 路径，而 LMCached-DRAM-LW 表示采用逐层内存拷贝与重叠的 DRAM 路径。

4.2.1 检索与存储的带宽性能。 为了隔离存储子系统的性能特征，我们跳过模型执行流水线，直接对 `retrieve` 和 `store` 接口的原始带宽进行基准测试。所有基于 SSD 的后端均采用双盘 RAID-0 配置进行评估。评估覆盖了从 1K 到 128K token 的多种序列长度，涵盖四个具有代表性的存储后端。随着前缀长度增加，检索带宽成为主导性能因子，如图 9(a) 所示。LMCached-DRAM 表现显著不稳定——例如，在 16K token 时，由于内存碎片化开销，其吞吐量下降至 8.5 GB/s。相比之下，Tutti 保持平滑的近线性缩放趋势，对于更长的上下文可达到高达 25.9 GB/s 的性能。与 LMCached-GDS 相比，后者即使使用两块 SSD 性能也饱和在约 11.9 GB/s，Tutti 的检索带宽最高可达其 2.08× 倍。

图 9(b) 报告了存储带宽。尽管 LMCached-DRAM 由于内存内写入而自然达到最高的原始带宽（最高可达 18.4 GB/s），但其缺乏持久性且受限于 DRAM 容量。在持久化存储后端中，Tutti 始终优于 LMCached-SSD 和 LMCached-GDS：它在所有测试长度下均维持约 10 GB/s 的写入带宽

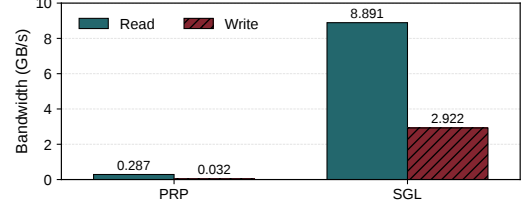


图 10. 单线程读/写微基准测试下的 PRP 与 SGL 带宽对比。相较于 PRP，SGL 在读取和写入带宽上均有显著提升。

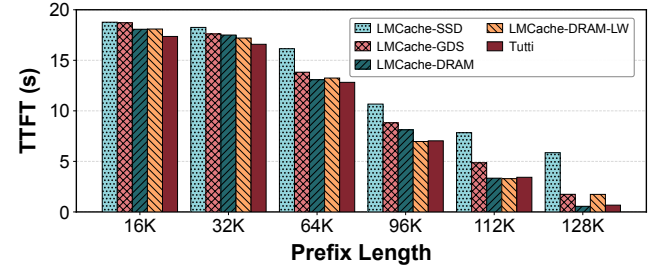


图 11. Llama3-8B-Instruct（单 GPU）上不同前缀长度下的 TTFT 性能对比。Tutti 展现出更优的 I/O 效率，相比 LMCached-GDS，TTFT 最高降低 61.4%。

（例如，在 128K token 时为 9.8 GB/s），而即使使用相同的双 SSD 配置，LMCached-GDS 的带宽仍保持在约 7 GB/s。值得注意的是，Tutti 的性能受存储设备本身的限制，因为每个 SSD 的峰值顺序写入带宽不超过 10 GB/s。先前的工作 [41] 表明，对于端到端推理性能而言，存储带宽的重要性低于检索带宽，而 10 GB/s 的带宽在大多数场景下已足以维持高性能。

4.2.2 PRP 与 SGL 带宽。 为了验证在我们的设计中应用 SGL 的影响，我们运行了一个单 GPU 线程的微基准测试，每次操作读写 500 MB 的数据。如图 10 所示，在 PRP 下，读/写带宽分别为 0.287 GB/s 和 0.032 GB/s，而切换到 SGL 后提升至 8.891 GB/s 和 2.922 GB/s，分别实现了 31.0× 和 91.3× 的提升。其关键原因是，与 PRP 相比，SGL 命令减少了主机与 NVMe 设备之间的 PCIe 通信开销，降低了命令/描述符处理开销，并稳定了队列进度。

4.2.3 不同上下文长度下的 TTFT 性能。 We evaluate TTFT under varying prefix reuse by fixing the total input length to 128k tokens and increasing the cached prefix from 16k to 128k. LMCached-SSD suffers severe degradation under high reuse due to limited bandwidth; at a 112k prefix, its TTFT rises to 7.84s. In contrast, our system sustains stable performance by overlapping retrieval with the remaining computation, achieving 3.43s at the same prefix—2.28× faster than SSD. Compared to LMCached-GDS, our method consistently maintains an advantage across all prefix lengths, with improvements ranging from 5.8% at 32k up to 61.4% at 128k. Notably, for moderate reuse (16k–96k), our system

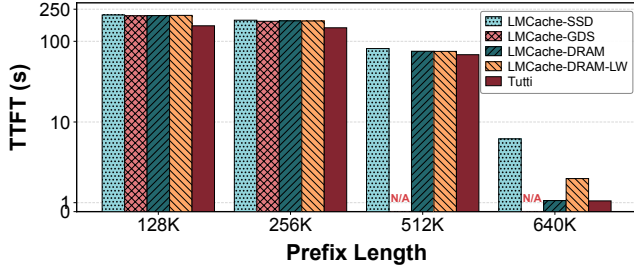


图 12. GLM-4-9B-1M 的分布式可扩展性 (2-GPU, 4-磁盘)。Tutti 通过避免暂存缓冲区开销, 克服了 LMCached-GDS 在 512K/640K 时的内存溢出失败问题, 展现出架构的鲁棒性, 并在 640K 时实现了最佳的 TTFT 1.2 秒。

even matches or exceeds DRAM performance, achieving up to 13.4% improvement—indicating that effective I/O–compute overlap can outweigh DRAM’s raw latency. Only in extremely high reuse conditions ($>96k$), where the workload becomes almost purely retrieval-bound, does DRAM regain its expected lead, with our system trailing by at most 20.6%.

4.2.4 多 GPU 可扩展性. 为了评估 Tutti 在分布式环境中的可扩展性, 我们在两个 GPU (分别位于不同的 PCIe 根复合体下) 和四个磁盘 (每个 GPU 的根复合体连接两个磁盘) 上测试了使用 GLM-4-9B-Chat-1M 模型的 TTFT 性能。GPU 之间通过 NVLink 连接。

实验数据凸显了 Tutti 的卓越性能: 在前缀长度为 128K 时, Tutti 的 TTFT 仅需 155.743 秒, 相比 LMCached-GDS (207.12 秒) 降低了约 25% 的延迟。在更长上下文场景下, LMCached-GDS 暴露出其依赖 GDS 技术带来的关键局限性。GDS 利用 cufile 实现存储到 GPU 内存的直接数据传输。关键在于, 为了在推理任务之外启用该机制, cufile 必须分配一定数量的 GPU 内存作为暂存缓冲区。在长序列推理过程中, 这种用于 I/O 加速的内存分配迅速超过了可用的 GPU 内存容量, 触发了致命的内存不足 (OOM) 错误。因此, LMCached-GDS 在 512K 和 640K 的测试中均未能完成 (标记为 N/A)。相比之下, Tutti 与推理引擎深度集成, 并提供寄存器接口, 可直接管理 GPU 内存, 无需中间暂存缓冲区。这使得 Tutti 成功完成了最具挑战性的测试, 在极端 640K 前缀长度下最终实现了整体最优的 TTFT 1.2 秒。我们认为, 高性能 I/O 必须与计算深度集成并协同优化, 而不应被简单视为一个第三方插件。

4.2.5 逐层异步流水线对比. 为了验证 Slack-Aware I/O 调度器的有效性, 我们将总推理延迟分解为计算时间和气泡时间。该评估对比了三种不同存储后端的性能特征, 它们均采用逐层流水线策略。我们在此延迟分解研究中特别排除了 LMCached-GDS, 因为其当前实现不支持该策略。通过将提示长度固定在 32K 并改变命中率, 我们调节了计算与加载的比率。核心原理是实现数据传输与逐层计算之间的深度重叠。理想情况下, 只要某一层的计算时间超过其数据传输时间 ($T_{compute} > T_{transfer}$), 传输延迟即可被完全掩码, 从而实现接近零的气泡时间。

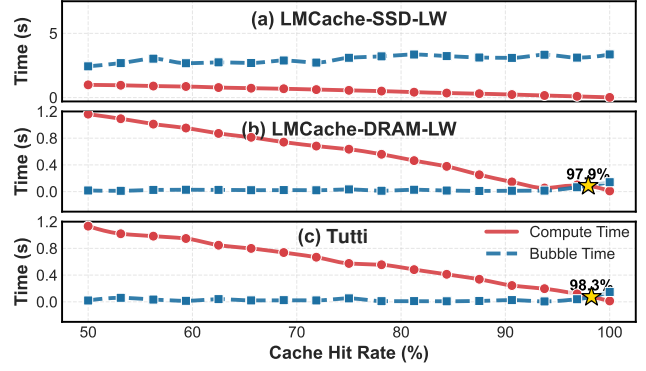


图 13. 按缓存命中率分解延迟, 突出显示气泡时间开始超过计算时间的临界交叉点 (*). 我们的逐层异步机制成功将这一临界点推至极高的缓存命中率 98.3%, 在测试值域内保持接近最优的计算密集型状态。

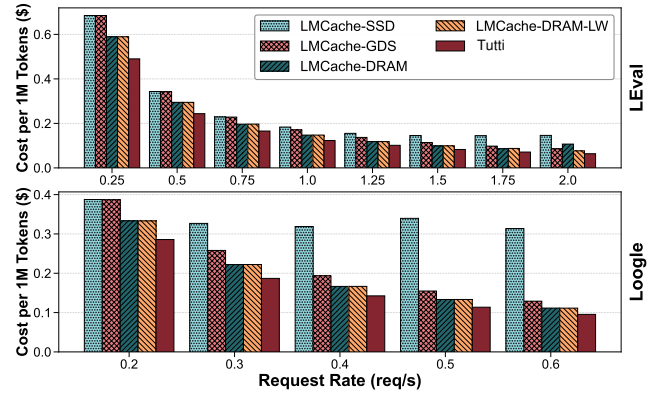


图 14. 在 LEval 和 LooGLE 工作负载下, 每百万 token 的推理成本。Tutti 通过利用固态硬盘 (SSDs) 实现了最低成本。

如图 13(a) 所示, LMCached-SSD 的流水线气泡过大; 无法被较短的计算阶段有效隐藏。低效的流水线暴露了原始的传输延迟, 导致显著的气泡时间 (蓝色虚线), 并降低了端到端性能。相比之下, 图 13(c) 表明, 我们的系统成功掩码了传输开销。在大部分测试值域内, 气泡时间可忽略不计 (平均约 25ms), 在 93.75% 命中率时进一步降至仅 6ms。Tutti 保持计算密集型特征 (由红色实线主导), 实现了接近最优的执行曲线, 与图 13(b) 中基于 DRAM 的强基准表现相似。我们进一步识别出一个关键的“交叉点” (以星号标记), 该点标志着从计算密集型向 I/O 密集型状态的转移。至关重要的是, 对于我们的系统而言, 这一交叉点被推至极高的缓存命中率 98.3%, 相比 LMCached-SSD 基准所观察到的明显更低阈值, 具有显著提升。这一结果明确证明, 我们的分层机制成功将“有效零气泡区”扩展至其物理极限, 仅在计算极度稀疏时引入微小气泡。

4.3 推理成本

为了量化 Tutti 的经济收益，我们计算了以 token 生成吞吐量归一化的服务成本。总体代价包括 GPU 费用和分层存储架构（DRAM 和 SSD）的开销。公式定义如下：

$$\text{Cost}_{1M} = \frac{\overbrace{P_{GPU} \cdot N_{GPU}}^{\text{Compute Cost}} + \overbrace{P_{mem} \cdot S_{mem} + P_{ssd} \cdot S_{ssd}}^{\text{Storage Cost}}}{\text{Throughput (tokens/hour)}} \times 10^6 \quad (1)$$

其中 P_{GPU} 为每小时 GPU 价格， N_{GPU} 为 GPU 数量， P_x/S_x 分别表示 DRAM/SSD 的单价和容量。

我们采用典型的云服务定价：每块 NVIDIA H100 GPU 每小时 5 美元，DRAM 每 GB 每小时 0.0088 美元，NVMe SSD 每 GB 每小时 0.000082 美元 [3, 4]。图 14 展示了 LEval 和 LooGLE 工作负载下每百万 token 的成本。Tutti 在所有请求速率下始终展现出最优的成本效益表现。随着上下文长度增加，基于 DRAM 的系统被迫为 KV 缓存配置更大的内存容量，从而导致显著升高的运行成本。相比之下，Tutti 将大部分 KV 数据卸载至 SSD（其单位 GB 成本约为 DRAM 的 100×）。尽管 LMCache-SSD 同样利用了这种高性价比的存储介质，但其固有的性能开销限制了吞吐量。该低效性导致 GPU 利用率不足，从而实质上抬高了单位成本。相比之下，我们的系统充分饱和 GPU 计算资源，最大化吞吐量，并优化每 GPU 小时所产生的 token 数量。具体而言，在 LooGLE 工作负载、0.5 QPS 条件下，Tutti 相较于 LMCache-SSD 降低了 66.2% 的服务成本，并较 LMCache-GDS 提升约 27%。

5 结论与未来工作

在本文中，我们提出了 Tutti，一种以 GPU 为中心、基于 SSD 的 KV 缓存存储系统，用于长上下文大语言模型服务。Tutti 从 GPU HBM 与 NVMe SSD 之间的关键数据和 I/O 控制路径中移除了 CPU 干预。通过结合以 GPU 为中心的对象存储设计与分层式 GPU 计算-I/O 流水线，Tutti 使基于 SSD 的 KV 缓存实现了接近 DRAM 的效率，同时有效抑制了 GPU 停顿时间。我们的评估表明，在严格的 SLO 约束下，与当前最先进的支持 GDS 的基于 SSD 的解决方案相比，Tutti 将 TTFT 降低了 78.3%，并将可实现的请求速率提升了 2×。Tutti 还使大语言模型服务成本降低了约 27%。

Acknowledgments

感谢华中科技大学的陈梦磊在本研究早期阶段对 GPU 哈希方面的宝贵指导。同时，我们也感谢武汉大学的张铮在 GPU I/O 内核性能分析方面给予的指导与帮助。

References

- [1] Aliyun. 2025. PolarKVCache. <https://help.aliyun.com/zh/polaradb/polaradb-for-mysql/user-guide/polar-kvcache-inference-acceleration>.
- [2] Chenxin An, Shansan Gong, Ming Zhong, Xingjian Zhao, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. 2023. L-Eval: Instituting Standardized Evaluation for Long Context Language Models. *arXiv:2307.11088* [cs.CL] <https://arxiv.org/abs/2307.11088>
- [3] AWS. 2025. Amazon ec2 p4d pricing. <https://aws.amazon.com/ec2/instance-types/p4/>.
- [4] AWS. 2025. Amazon ec2 pricing. <https://aws.amazon.com/ec2/pricing/>.
- [5] Chia-Hao Chang, Jihoon Han, Anand Sivasubramaniam, Vikram Sharma Mailthody, Zaid Qureshi, and Wen-Mei Hwu. 2024. GMT: GPU Orchestrated Memory Tiering for the Big Data Era. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 464–478.
- [6] Weijian Chen, Shuibing He, Haoyang Qu, Ruidong Zhang, Siling Yang, Ping Chen, Yi Zheng, Baoxing Huai, and Gang Chen. 2025. IMPRESS: An Importance-Informed Multi-Tier Prefix KV Storage System for Large Language Model Inference. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 187–201.
- [7] Deepseek. 2025. Models and Pricing. https://api-docs.deepseek.com/quick_start/pricing/.
- [8] DeepSpeedAI. 2025. DeepNVMe: Affordable I/O scaling for Deep Learning Applications. <https://github.com/deepspeedai/DeepSpeed/blob/master/blogs/deepnvme/06-2025/README.md/>.
- [9] Devansh. 2026. How Weka is Solving AI’s Trillion Dollar Memory Problem. https://www.artificialintelligencemadesimple.com/p/how-one-startup-is-breaking-nvidias?utm_source=publication-search.
- [10] Diego Didona, Jonas Pfefferle, Nikolas Ioannou, Bernard Metzler, and Animesh Trivedi. 2022. Understanding modern storage APIs: a systematic study of libaio, SPDK, and io_uring. In *Proceedings of the 15th ACM International Conference on Systems and Storage*. 120–127.
- [11] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Attentionstore: Cost-effective attention reuse across multi-turn conversations in large language model serving. *arXiv preprint arXiv:2403.19708* 52 (2024), 20–38.
- [12] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 111–126.
- [13] Shiwei Gao, Youmin Chen, and Jiwu Shu. 2025. Fast state restoration in llm serving with hcache. In *Proceedings of the Twentieth European Conference on Computer Systems*. 128–143.
- [14] Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Dan Zhang, Diego Rojas, Guanyu Feng, Hanlin Zhao, et al. 2024. Chatglm: A family of large language models from glm-130b to glm-4 all tools. *arXiv preprint arXiv:2406.12793* (2024).
- [15] Yang Hu, Hong Jiang, Dan Feng, Hao Luo, and Lei Tian. 2015. PASS: A proactive and adaptive SSD buffer scheme for data-intensive workloads. In *2015 IEEE International Conference on Networking, Architecture and Storage (NAS)*. IEEE, 54–63.
- [16] Jinwoo Jeong and Jeongseob Ahn. 2025. Accelerating LLM Serving for Multi-turn Dialogues with Efficient Resource Management. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1–15.
- [17] Chaoyi Jiang, Lei Gao, Hossein Entezari Zarch, and Murali Annavaram. 2024. KVPR: Efficient LLM Inference with I/O-Aware KV Cache Partial Recomputation. *arXiv preprint arXiv:2411.17089* (2024).
- [18] KIOXIA. 2025. KIOXIA CM7-V Series Enterprise NVMe™ Mixed Use SSD. <https://apac.kioxia.com/en-apac/business/ssd/enterprise-ssd/cm7-v.html>.
- [19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

- [20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [21] Hyungwoo Lee, Kihyun Kim, Jinwoo Kim, Jungmin So, Myung-Hoon Cha, Hong-Yeon Kim, James J Kim, and Youngjae Kim. 2025. Disk-Based Shared KV Cache Management for Fast Inference in Multi-Instance LLM RAG Systems. In *2025 IEEE 18th International Conference on Cloud Computing (CLOUD)*. IEEE, 199–209.
- [22] Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. 2024. LooGLE: Can Long-Context Language Models Understand Long Contexts? arXiv:2311.04939 [cs.CL] <https://arxiv.org/abs/2311.04939>
- [23] Shaobo Li, Yirui Eric Zhou, Yuqi Xue, Yuan Xu, and Jian Huang. 2025. Managing Scalable Direct Storage Accesses for GPUs with GoFS. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 979–995.
- [24] Zejia Lin, Hongxin Xu, Guanyi Chen, Xianwei Zhang, and Yutong Lu. 2025. Bullet: Boosting GPU Utilization for LLM Serving via Dynamic Spatial-Temporal Orchestration. *arXiv preprint arXiv:2504.19516* (2025).
- [25] Renping Liu, Zhenhua Tan, Linbo Long, Yu Wu, Yujuan Tan, and Duo Liu. 2022. Improving fairness for SSD devices through DRAM over-provisioning cache management. *IEEE Transactions on Parallel and Distributed Systems* 33, 10 (2022), 2444–2454.
- [26] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. CacheGen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.
- [27] Vikram Sharma Mailthody. 2025. Advancing Memory and Storage Architectures for Next-Gen AI Workloads. *Flash Memory Summit* (2025), 1–27.
- [28] Jonas Markussen, Lars Bjørlykke Kristiansen, Pål Halvorsen, Halvor Kielland-Gyrud, Håkon Kvale Stensland, and Carsten Griwodz. 2021. Smartio: Zero-overhead device sharing through pcie networking. *ACM Transactions on Computer Systems (TOCS)* 38, 1-2 (2021), 1–78.
- [29] Meta. 2025. The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- [30] Meta AI. 2024. Meta-Llama-3-8B-Instruct. <https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>. Accessed: 2025-12-10.
- [31] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an llm to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [32] NVIDIA. 2025. TensorRT LLM’s Documentation. <https://nvidia.github.io/TensorRT-LLM/>.
- [33] Nvidia. 2024. NVIDIA GPUDirect Storage. <https://docs.nvidia.com/gpudirect-storage/index.html>.
- [34] NVIDIA. 2025. Cuda-programming-guide Green Contexts. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/green-contexts.html/>.
- [35] Doug O’Laughlin. 2026. Another Conversation with Val Bercovici Memory Markets. <https://www.fabricatedknowledge.com/p/another-conversation-with-val-bercovici>.
- [36] OpenAI. 2025. API Pricing. <https://openai.com/api/pricing/>.
- [37] Xiurui Pan, Endian Li, Qiao Li, Shengwen Liang, Yizhou Shan, Ke Zhou, Yingwei Luo, Xiaolin Wang, and Jie Zhang. 2025. InstAttention: In-Storage Attention Offloading for Cost-Effective Long-Context LLM Inference. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1510–1525.
- [38] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: Kimi’s KVCache-centric Architecture for LLM Serving. *arXiv preprint arXiv:2407.00079* (2024).
- [39] Shi Qiu, Weinan Liu, Yifan Hu, Jianqin Yan, Zhirong Shen, Xin Yao, Renhai Chen, Gong Zhang, and Yiming Zhang. 2025. GeminiFS: A Companion File System for GPUs. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 221–236.
- [40] Zaid Qureshi, Vikram Sharma Mailthody, Isaac Gelado, Seungwon Min, Amna Masood, Jeongmin Park, Jinjun Xiong, Chris J Newburn, Dmitri Vainbrand, I-Hsin Chung, et al. 2023. GPU-initiated on-demand high-throughput storage access in the BaM system architecture. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 325–339.
- [41] Zebin Ren, Krijn Doekemeijer, Tiziano De Matteis, Christian Pinto, Radu Stoica, and Animesh Trivedi. 2025. An I/O Characterizing Study of Offloading LLM Models and KV Caches to NVMe SSD. In *Proceedings of the 5th Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems*. 23–33.
- [42] SGLang. 2024. SGLang. <https://github.com/sgl-project/sglang?tab=readme-ov-file>.
- [43] Ao Shen, Zhiyao Li, and Mingyu Gao. 2024. Fastswitch: Optimizing context switching efficiency in fairness-aware large language model serving. *arXiv preprint arXiv:2411.18424* (2024).
- [44] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.
- [45] solidigm. 2024. Solidigm D7-PS1010. <https://www.solidigm.com/products/data-center/d7/ps1010.html>.
- [46] Solidigm. 2025. Solidigm D7-PS1010. <https://www.solidigm.com/products/data-center/d7/ps1010.html#configurator>.
- [47] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530* (2024).
- [48] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [49] NVM Express Workgroup. 2022. NVM Express Base Specification Revision 2.0c. <https://nvmexpress.org/wp-content/uploads/NVM-Express-Base-Specification-2.0c-2022.10.04-Ratified.pdf>.
- [50] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. 2025. Strata: Hierarchical Context Caching for Long Context Language Model Serving. *arXiv preprint arXiv:2508.18572* (2025).
- [51] Xie, Zhiqiang. 2025. SGLang HiCache: Fast Hierarchical KV Caching with Your Favorite Storage Backends. <https://lmsys.org/blog/2025-09-10-sglang-hicache/>.
- [52] Jianqin Yan, Shi Qiu, Yina Lv, Yifan Hu, Hao Chen, Zhirong Shen, Xin Yao, Renhai Chen, Jiwu Shu, Gong Zhang, et al. 2025. Phoenix: A Refactored I/O Stack for GPU Direct Storage without Phony Buffers. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1267–1283.
- [53] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng,

- Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
- [54] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) (*EuroSys '25*). Association for Computing Machinery, New York, NY, USA, 94–109. doi:10.1145/3689031.3696098
- [55] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. *arXiv preprint arXiv:2402.15220* (2024).
- [56] Zihao Yi, Jiarui Ouyang, Yuwen Liu, Tianhao Liao, Zhe Xu, and Ying Shen. 2024. A survey on recent advances in llm-based multi-turn dialogue systems. *arXiv preprint arXiv:2402.18013* (2024).
- [57] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.